

# **Diplomatura en programación web full stack con React JS**

Módulo 5:

# **Base de datos, integración con Node.js**

Unidad 2:

## **Introducción al lenguaje SQL**



## Presentación

En esta unidad vemos el lenguaje SQL, utilizado en bases de datos tanto para realizar consultas como para escribir o eliminar datos.



## Objetivos

Que los participantes logren...

- Hacer consultas a la base de datos.
- Escribir y modificar datos en las tablas.
- Eliminar registros de nuestra base de datos.



## Bloques temáticos

1. ¿Qué es?
2. Consultas.
3. Agregar, Modificar o Eliminar registros.

# 1. ¿Qué es?

El lenguaje de consulta estructurado o SQL (por sus siglas en inglés Structured Query Language) es un **lenguaje declarativo de acceso a bases de datos relacionales** que permite especificar diversos tipos de operaciones en ellas.

El SQL es un lenguaje de acceso a bases de datos que explota la flexibilidad y potencia de los sistemas relacionales y permite así gran variedad de operaciones.

## Características generales de SQL

Es un lenguaje declarativo de "alto nivel" o "de no procedimiento" que, gracias a su fuerte base teórica y su orientación al manejo de conjuntos de registros —y no a registros individuales— permite una alta productividad en codificación y la orientación a objetos. De esta forma, una sola sentencia puede equivaler a uno o más programas que se utilizarían en un lenguaje de bajo nivel orientado a registros. SQL también tiene las siguientes características:

- **Lenguaje de definición de datos:** El LDD de SQL proporciona comandos para la definición de esquemas de relación, borrado de relaciones y modificaciones de los esquemas de relación.
- **Lenguaje interactivo de manipulación de datos:** El LMD de SQL incluye lenguajes de consultas basado tanto en álgebra relacional como en cálculo relacional de tuplas.
- **SQL incorporado y dinámico:** Esto quiere decir que se pueden incorporar instrucciones de SQL en lenguajes de programación como: C++, C, Java, PHP, COBOL, Pascal y Fortran.
- **Autorización:** El LDD incluye comandos para especificar los derechos de acceso a las relaciones y a las vistas.

## Tipos de datos

- **Varchar:** Recibe cadena de palabras compuestas de letras, números y caracteres especiales.
- **Int** es el principal tipo de datos de valores enteros de SQL Server. Con números enteros con o sin signo
- **Date:** una fecha de calendario que contiene el año (de cuatro cifras), el mes y el día.
- **Time:** La hora del día en horas minutos segundos (el valor predeterminado es 0).

## 2. Consultas

Es el lenguaje de consulta universal para bases de datos.

Los mandatos de SQL se dividen en tres grandes grupos diferenciados.

- **DDL (Data Definition Language):** es el encargado de la definición de Bases de Datos, tablas, vistas e índices entre otros.
- **DML (Data Manipulation Language):** cuya misión es la manipulación de datos. A través de él podemos seleccionar, y actualizar insertar, eliminar datos.
- **DCL (Data Control Language):** encargado de la seguridad de la base de datos, en todo lo referente al control de accesos y privilegios entre los usuarios.

### Lenguaje de definición de datos de mysql

Para **crear una base de datos** es la consulta es la siguiente: CREATE DATABASE, se puede escribir en mayúscula o minúscula.



```
CREATE DATABASE NombreDeBaseDeDatos;
```

Para **crear una tabla**:

```
CREATE TABLE 'alumnos' ('nombre' VARCHAR(30), 'apellido'  
VARCHAR(30), 'FechaNac' DATETIME, 'email' VARCHAR(30));
```

En primer momento colocamos el nombre de la tabla en este caso alumnos y entre paréntesis se debe especificar el nombre y el tipo de dato de cada columna de la tabla.

Para **seleccionar datos de una tabla** y devolver un resultado, que es una matriz de datos (filas y columnas).

```
SELECT nombre_de_columna(s)  
FROM nombre_de_tabla
```

Para seleccionar los datos de todas las columnas de una tabla, se usa un asterisco en lugar de los nombres de las columnas:

```
SELECT *  
FROM nombre_de_la_tabla
```

Para seleccionar datos de las tablas en donde sólo queramos recuperar registros en donde una columna tenga un valor en particular, usamos la **cláusula WHERE**.



```
SELECT nombre_de_columna(s)
FROM nombre_de_tabla
WHERE nombre_de_columna operador valor
```

Si se quisiera recuperar registros que cumplan con más de una condición, se utiliza AND u OR entre las condiciones del WHERE:



```
SELECT nombre_de_columna(s)
FROM nombre_de_tabla
WHERE nombre_de_columna operador valor
AND|OR nombre_de_columna operador valor
```

### Ejemplos:



```
SELECT Nombre, Apellido
FROM alumnos
```

| Nombre   | Apellido |
|----------|----------|
| Laura    | Gomez    |
| Victoria | Sanchez  |

```
SELECT sueldo  
FROM empleados  
WHERE Puesto='Puesto1'
```

| Sueldo |
|--------|
| 20000  |

## Operadores que se pueden usarse con WHERE:

| Operador | Descripción                                                                                                                                                                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =        | Igual a                                                                                                                                                                                                                                                                                               |
| <>       | Diferente de                                                                                                                                                                                                                                                                                          |
| >        | Mayor que                                                                                                                                                                                                                                                                                             |
| <        | Menor que                                                                                                                                                                                                                                                                                             |
| >=       | Mayor que o igual a                                                                                                                                                                                                                                                                                   |
| <=       | Menor que o igual a                                                                                                                                                                                                                                                                                   |
| BETWEEN  | Entre cierto rango                                                                                                                                                                                                                                                                                    |
| LIKE     | Busca cierto patrón de caracteres en los datos. Al patrón buscado se lo precede o sucede con “%” según se desee:<br>LIKE ‘%ma%’: Busca campos que contengan la sílaba “ma”.<br>LIKE ‘%ma’: Busca campos que terminen con la sílaba “ma”.<br>LIKE ‘ma%’: Busca campos que empiecen con la sílaba “ma”. |

## Consultas auxiliares

### ORDER BY

Para lograr un ordenamiento específico de la respuesta podemos utilizar la palabra clave “ORDER BY”



```
ORDER BY nombre_de_columna ASC | DESC
```

Para dar dos ordenamientos sucesivos (por edad y por apellido, por ejemplo), vamos agregando las instrucciones una detrás de la otra en el orden en que queramos que se apliquen:



```
SELECT Nombre, Apellido, Edad  
FROM Empleados  
ORDER BY Edad ASC, Apellido ASC
```

### LIMIT

Se utiliza para cuando sólo queremos una cantidad limitada de datos



```
SELECT nombre_de_columna(s)
FROM nombre_de_tabla
LIMIT numero_fila_inicial, numero_de_filas
```

**número\_fila\_inicial** es desde qué fila de las que nos devolvería queremos empezar

**número\_de\_filas** es el es la cantidad de filas que queremos recibir



```
SELECT ID_Empleado, Nombre, Apellido
FROM Empleados
LIMIT 2,1
```

| ID_Empleado | Nombre | Apellido |
|-------------|--------|----------|
| 2           | Laura  | Conti    |
| 3           | Juana  | Diaz     |

## 3. Agregar, Modificar o Eliminar registros

El **lenguaje de manipulación de datos** consiste en 3 consultas para manipular los datos.

### INSERT

Inserta nuevos registros en una tabla



```
INSERT INTO nombre_de_tabla  
VALUES (valor1, valor2,....)
```



```
INSERT INTO Empleados VALUES (03, 'Carlos', 'García');
```

| ID_Empleado | Nombre | Apellido |
|-------------|--------|----------|
| 1           | Laura  | Conti    |
| 2           | Juana  | Diaz     |
| 3           | Carlos | García   |

O también se pueden especificar las columnas a las que se les agrega datos:



```
INSERT INTO nombre_de_tabla (columna1, columna2,...)
VALUES (valor1, valor2,....);
```



```
INSERT INTO Empleados (ID_Empleado, Nombre, Apellido) VALUES (03,
'Carlos', 'García');
```

## DELETE

Sirve para eliminar registros de una tabla.



```
DELETE FROM nombre_de_tabla
WHERE nombre_de_columna operador valor
```



```
DELETE FROM Empleados
WHERE ID_Empleado=2
```

| ID_Empleado | Nombre | Apellido |
|-------------|--------|----------|
| 1           | Laura  | Conti    |
| 3           | Carlos | García   |

También se pueden eliminar todas las filas de una tabla:

```
DELETE FROM nombre_de_la_tabla
```

## UPDATE

Sirve para modificar datos de una tabla

```
UPDATE nombre_de_tabla  
SET nombre_de_columna=valor  
WHERE nombre_de_columna operador valor
```



```
UPDATE Empleados  
SET nombre=Lara  
WHERE ID_Empleado= 1;
```

| ID_Empleado | Nombre | Apellido |
|-------------|--------|----------|
| 1           | Lara   | Conti    |
| 3           | Carlos | García   |

## Conclusión

En resumen, SQL es un lenguaje de programación diseñado para gestionar y manipular bases de datos relacionales. Permite realizar diversas operaciones, como la creación, modificación, eliminación y consulta de datos.

## En la próxima unidad veremos...

Aplicaremos los conocimientos aprendidos de bases de datos y SQL a nuestros proyectos de Express y Node.js.



## Bibliografía utilizada y sugerida

### Artículos de revista en formato electrónico:

PostgreSQL Global Development Group. (s.f.). SQL commands. PostgreSQL Documentation. <http://www.postgresql.org/docs/9.1/static/sql-commands.html>

Stabare, M. (s.f.). Tutorial de SQL. Universidad Nacional de Colombia. <http://www.unalmed.edu.co/~mstabare/Sql.pdf>

Wampserver. (s.f.). Wampserver: servidor local de desarrollo web. <http://www.wampserver.es/>